# 0130. Surrounded Regions

Medium · Graphs

leetcode.com/problems/surrounded-regions

# Problem

- Flip every 'O' region that is fully surrounded by 'X' into 'X'. Regions touching the border survive.

# Intuition

- It is easier to find the survivors: flood-fill from every border 'O' and mark it safe ('S').
- Then every remaining 'O' is captured (→ 'X') and every 'S' is restored to 'O'.

# Brute Force

- For each region check whether it touches the border, with a BFS per region: same order but more bookkeeping.

# Optimal Approach

- DFS from each border 'O', marking 'S'.
- Sweep: 'O' → 'X', 'S' → 'O'.

# Worked Example

```
X X X X          X X X X
X O O X    ->    X X X X      inner region captured
X X O X          X X X X
X O X X          X O X X      border-connected O survives
```

# Java Solution

```java
public class SurroundedRegions {
    public void solve(char[][] b) {
        int m = b.length, n = b[0].length;
        for (int r = 0; r < m; r++) { mark(b, r, 0); mark(b, r, n - 1); }
        for (int c = 0; c < n; c++) { mark(b, 0, c); mark(b, m - 1, c); }
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) b[r][c] = b[r][c] == 'S' ? 'O' : 'X';
    }

    private void mark(char[][] b, int r, int c) {
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length || b[r][c] != 'O') return;
        b[r][c] = 'S';
        mark(b, r + 1, c); mark(b, r - 1, c); mark(b, r, c + 1); mark(b, r, c - 1);
    }
}
```

# Complexity

- Time: O(m  n). Space: O(m  n) recursion worst case.

# Pitfalls

- Do not flip during the first pass; mark first, flip later.